

Міністерство освіти і науки України  
Харківський національний університет радіоелектроніки

Кафедра програмної інженерії

Звіт  
З дисципліни «Аналіз та рефакторинг коду»  
З лабораторної роботи №3

Виконав:

ст. гр. ПЗП-22-9

Терновський Денис

Перевірив:

Дашенков Д. С.

Харків 2024

## 2 РОЗРОБКА БІЗНЕС-ЛОГІКИ ТА ФУНКЦІЙ АДМІНІСТРУВАННЯ СЕРВЕРНОЇ ЧАСТИНИ ПРОГРАМНОЇ СИСТЕМИ

### **Мета:**

Розробити бізнес-логіку та функції адміністрування серверної частини програмної системи.

### **Хід роботи:**

**Бізнес-логіка серверної частини програмної системи TaskLamp**

#### **1. Основні функції бізнес-логіки**

##### **1.1 Управління користувачами та їхніми даними:**

- **Реєстрація користувачів:**

Дані, що зберігаються: ім'я користувача, електронна пошта, пароль (хешований) і дата створення.

- **Редагування та оновлення:**

Користувачі можуть оновлювати ім'я, електронну пошту або пароль через API.

- **Авторизація:**

Перевірка логіна та пароля для доступу до системи. Реалізовано з використанням хешування паролів (bcrypt).

- **Видалення користувачів:**

Забезпечує можливість видалення облікових записів, якщо це необхідно.

##### **1.2 Управління логами пристроїв (DeviceLogs):**

- **Збереження даних про енергоспоживання:**

Збереження показників:

- Потужність у кімнаті 1 (Room1Power).
- Потужність у кімнаті 2 (Room2Power).
- Максимальна потужність (MaxPower), отримана з налаштувань пристрою.

- **Логуювання змін:**

Автоматичний запис оновлень споживання енергії в таблицю **logs** після кожного запиту оновлення пристрою.

- **Отримання логів:**

Можливість отримати повний список записів для конкретного пристрою або користувача з фільтрацією за DeviceID або UserID.

- **Видалення логів:**

Видалення певного запису на основі LogID.

### 1.3 Генерація звітів:

- **Звіти для аналізу даних:**
  - Тренди зміни виконаних завдань за певний період.
  - Відсоток виконання завдань користувачем.
  - Користувачі з найбільшою кількістю невиконаних завдань.
- **Звіти для енергоспоживання:**
  - Загальні витрати енергії за пристроями.
  - Перевищення максимальних значень потужності.

### 1.4 Управління сповіщеннями (Notifications):

- **Автоматичне створення сповіщень:**
  - У разі перевищення максимальної потужності (MaxPower).
  - У разі несправності пристрою або втрати даних.
- **Позначення як прочитаних:**

Можливість позначити сповіщення як прочитані через API.

- **Фільтрація сповіщень:**

Сортування сповіщень за статусом (IsRead) для зручності користувачів.

## 2. Реалізація CRUD-операцій

- **Користувачі (Users):**

Реалізовано створення, отримання, редагування та видалення даних.

- **Завдання (Tasks):**

CRUD-операції дозволяють працювати із завданнями користувачів, включаючи пошук за статусом, датою та іншими параметрами.

- **Статуси пристроїв (DeviceStatus):**

CRUD-операції для оновлення статусу пристроїв (активність, остання синхронізація).

- **Налаштування лампи (LampSettings):**

CRUD для збереження кольорів, яскравості та інших параметрів RGB LED стрічки.

### **3. Додаткові функції:**

- **Багатомовність:**

Відображення текстів і сповіщень відповідно до мови користувача (підтримка зберігається у таблиці Settings).

- **Налаштування:**

Збереження параметрів лампи та облікових записів у таблиці **settings**.

- **Інтеграція з мобільними пристроями:**

Мобільний додаток підтримує синхронізацію завдань, сповіщень і логів.

Ця бізнес-логіка забезпечує інтегровану систему управління, що відповідає потребам користувачів і дозволяє масштабувати TaskLamp у майбутньому.

## 1) Код реалізації звітів

```
const express = require('express');
const router = express.Router();
const { Task } = require('../models');
const { Op } = require('sequelize');

// Получить общую статистику по задачам пользователя
router.get('/user/:userId', async (req, res) => {
  try {
    const userId = req.params.userId;

    // Получить общее количество задач
    const totalTasks = await Task.count({ where: { userId } });

    // Выполненные задачи
    const completedTasks = await Task.count({
      where: { userId, isCompleted: true },
    });

    // Невыполненные задачи
    const pendingTasks = totalTasks - completedTasks;

    // Просроченные задачи
    const overdueTasks = await Task.findAll({
      where: {
        userId,
        dueDate: { [Op.lt]: new Date() },
        isCompleted: false,
      },
    });

    // Процент выполнения
    const completionRate = totalTasks > 0 ? (completedTasks / totalTasks) * 100 : 0;

    res.json({
      totalTasks,
      completedTasks,
      pendingTasks,
      overdueTasks: overdueTasks.map(task => ({
        id: task.id,
        title: task.title,
        dueDate: task.dueDate,
      })),
      completionRate: `${completionRate.toFixed(2)}%`,
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Получить список просроченных задач
```

```

router.get('/user/:userId/overdue', async (req, res) => {
  try {
    const userId = req.params.userId;

    const overdueTasks = await Task.findAll({
      where: {
        userId,
        dueDate: { [Op.lt]: new Date() },
        isCompleted: false,
      },
    });

    res.json(
      overdueTasks.map(task => ({
        id: task.id,
        title: task.title,
        dueDate: task.dueDate,
      })))
  );
} catch (error) {
  res.status(500).json({ error: error.message });
}
});

// Получить процент выполнения задач
router.get('/user/:userId/completion-rate', async (req, res) => {
  try {
    const userId = req.params.userId;

    const totalTasks = await Task.count({ where: { userId } });
    const completedTasks = await Task.count({
      where: { userId, isCompleted: true },
    });

    const completionRate = totalTasks > 0 ? (completedTasks / totalTasks) * 100 : 0;

    res.json({ completionRate: `${completionRate.toFixed(2)}%` });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;

```

## API для управління користувачами

### 1. Отримання всіх користувачів

- **Метод:** GET /users
- **Опис:** Отримання списку всіх зареєстрованих користувачів.
- **Параметри:** Немає.
- **Відповідь:**
  - Масив об'єктів, що містить інформацію про користувачів (ім'я, email, тощо).

### 2. Отримання даних користувача за ID

- **Метод:** GET /users/:id
- **Опис:** Отримання інформації про конкретного користувача за його ID.
- **Параметри:**
  - id — Унікальний ідентифікатор користувача.
- **Відповідь:**
  - Об'єкт з інформацією про користувача (ім'я, email).
  - Статус 404, якщо користувача не знайдено.

### 3. Створення нового користувача

- **Метод:** POST /users
- **Опис:** Додавання нового користувача до системи.
- **Тіло запиту:**

```
{  
  "username": "string",  
  "email": "string",  
  "password": "string"  
}
```

- **Відповідь:**
  - Об'єкт створеного користувача.
  - Статус 500 у разі помилки.

### 4. Оновлення даних користувача

- **Метод:** PUT /users/:id

- **Опис:** Редагування даних користувача.
- **Параметри:**
  - id — Унікальний ідентифікатор користувача.
- **Тіло запиту:**

```
{
  "username": "string",
  "email": "string",
  "password": "string"
}
```

- **Відповідь:**
  - Оновлені дані користувача.
  - Статус 404, якщо користувача не знайдено.

## 5. Видалення користувача

- **Метод:** DELETE /users/:id
- **Опис:** Видалення користувача за його ID.
- **Параметри:**
  - id — Унікальний ідентифікатор користувача.
- **Відповідь:**
  - Повідомлення про успішне видалення.
  - Статус 404, якщо користувача не знайдено.

## 6. Пошук користувачів за іменем

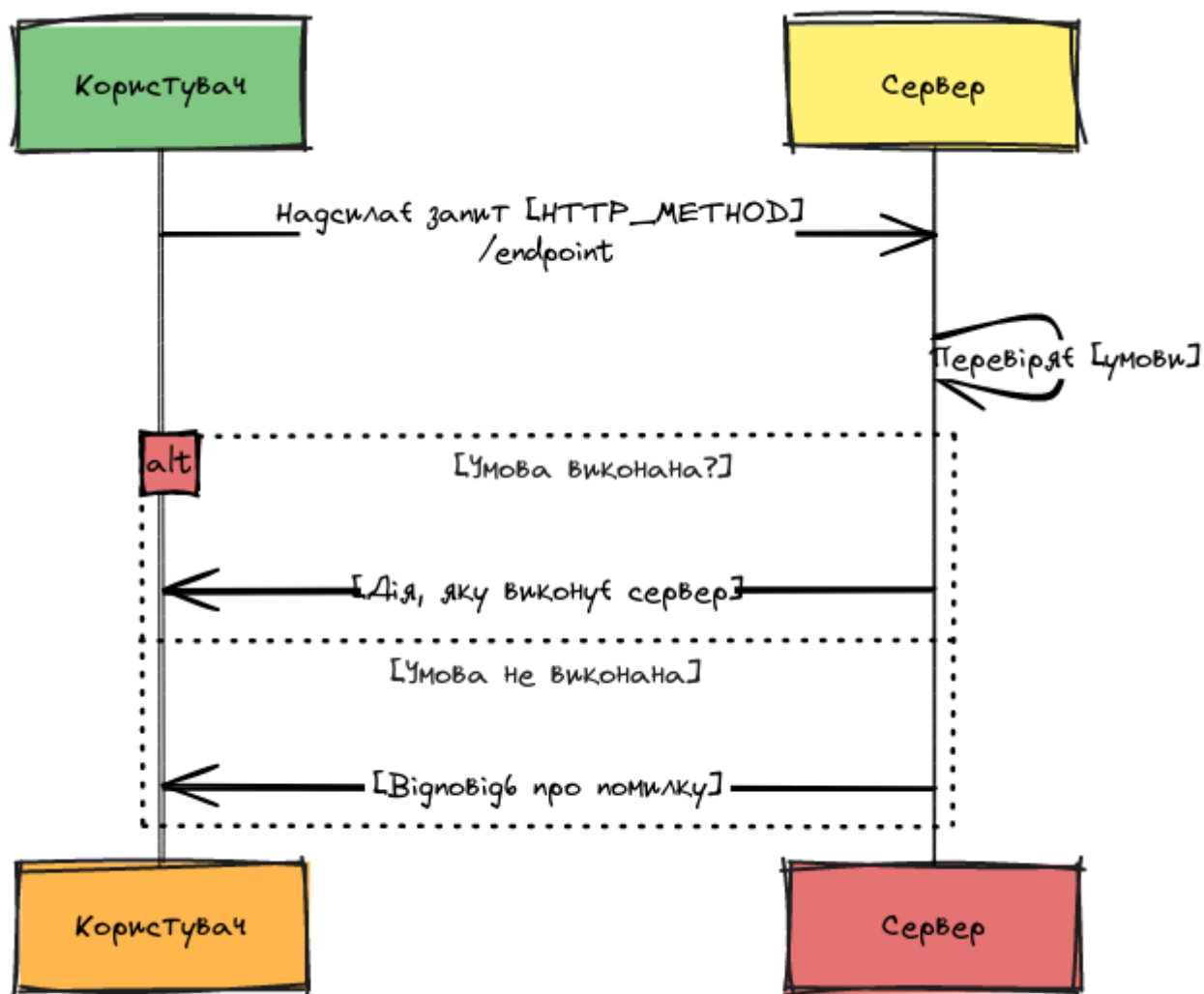
- **Метод:** GET /users/search/:username
- **Опис:** Пошук користувачів за їхнім ім'ям.
- **Параметри:**
  - username — Ім'я користувача.
- **Відповідь:**
  - Масив користувачів, що відповідають пошуковому запиту.
  - Статус 404, якщо користувачів не знайдено.

---

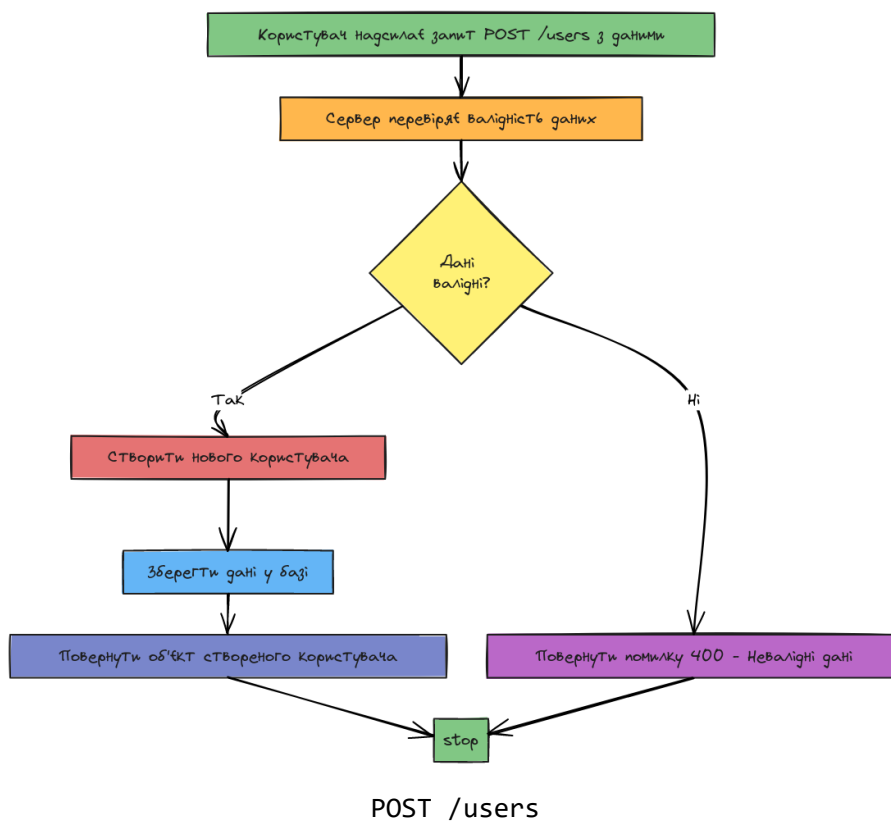
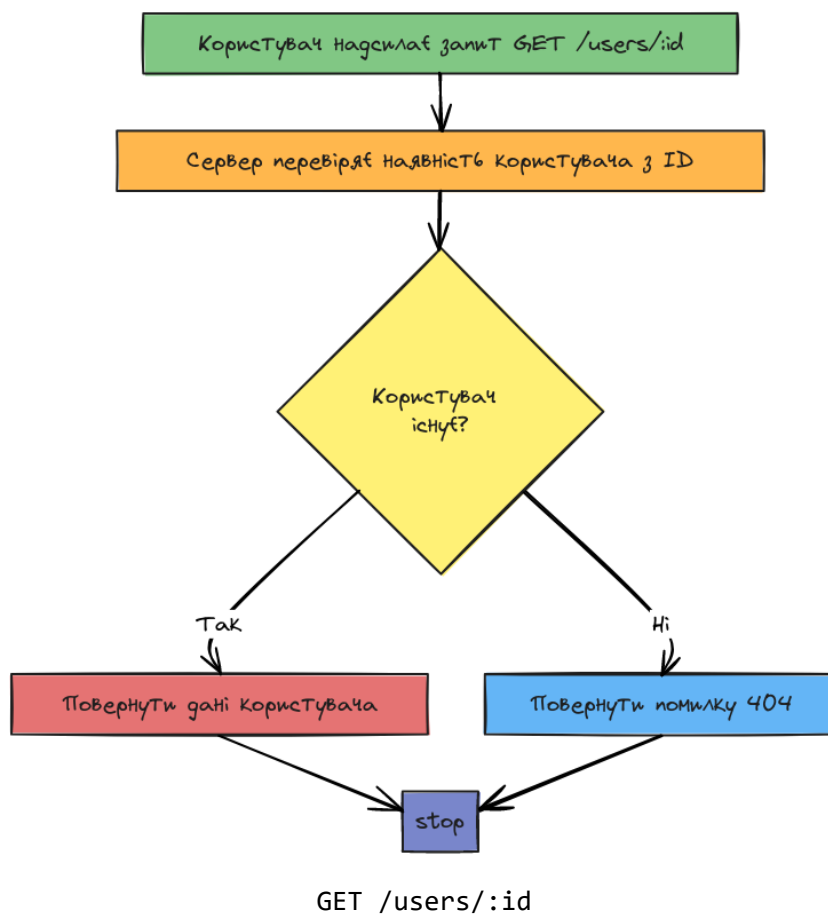
Ці ендпоінти забезпечують повний набір функцій для роботи з обліковими записами користувачів.

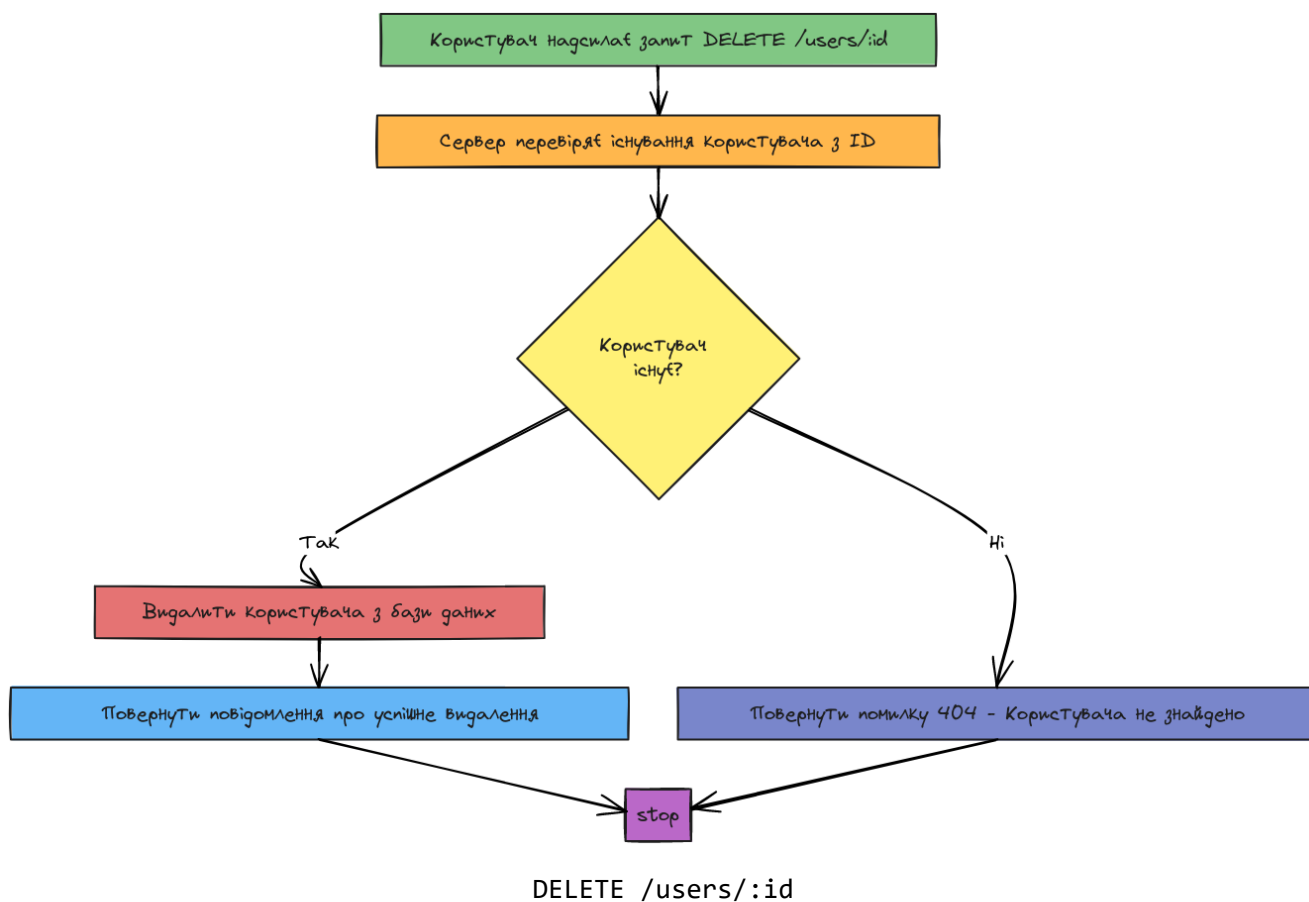
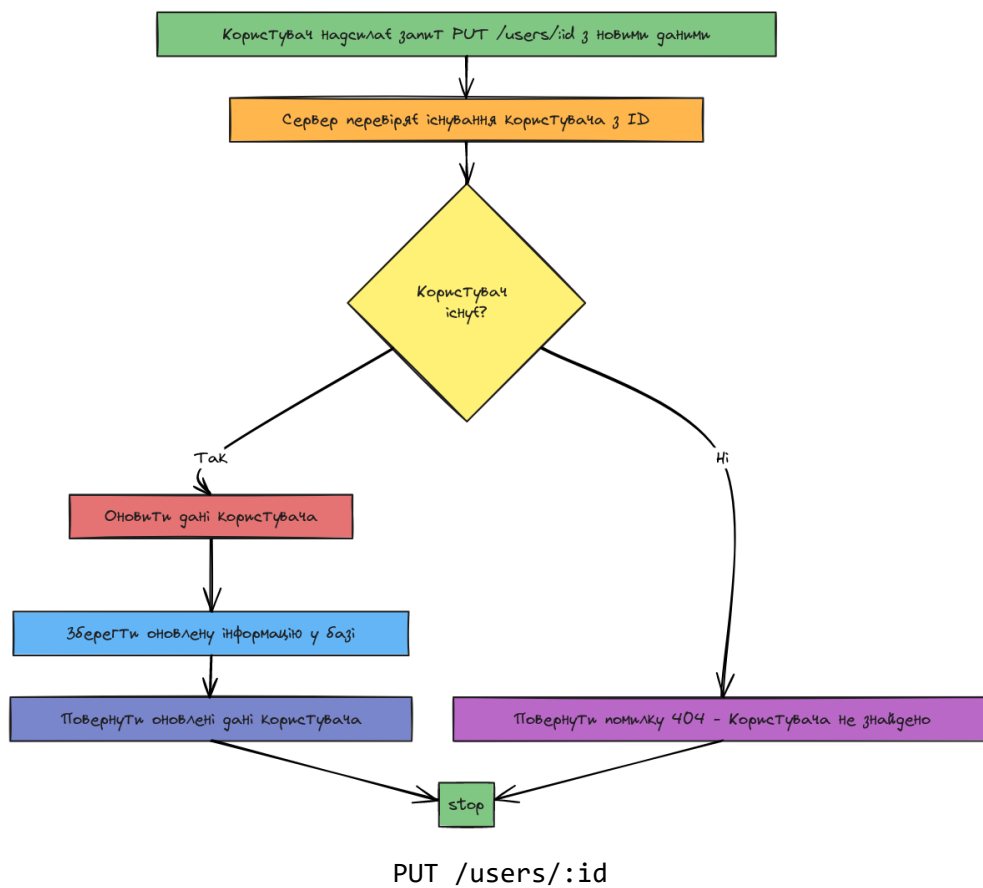


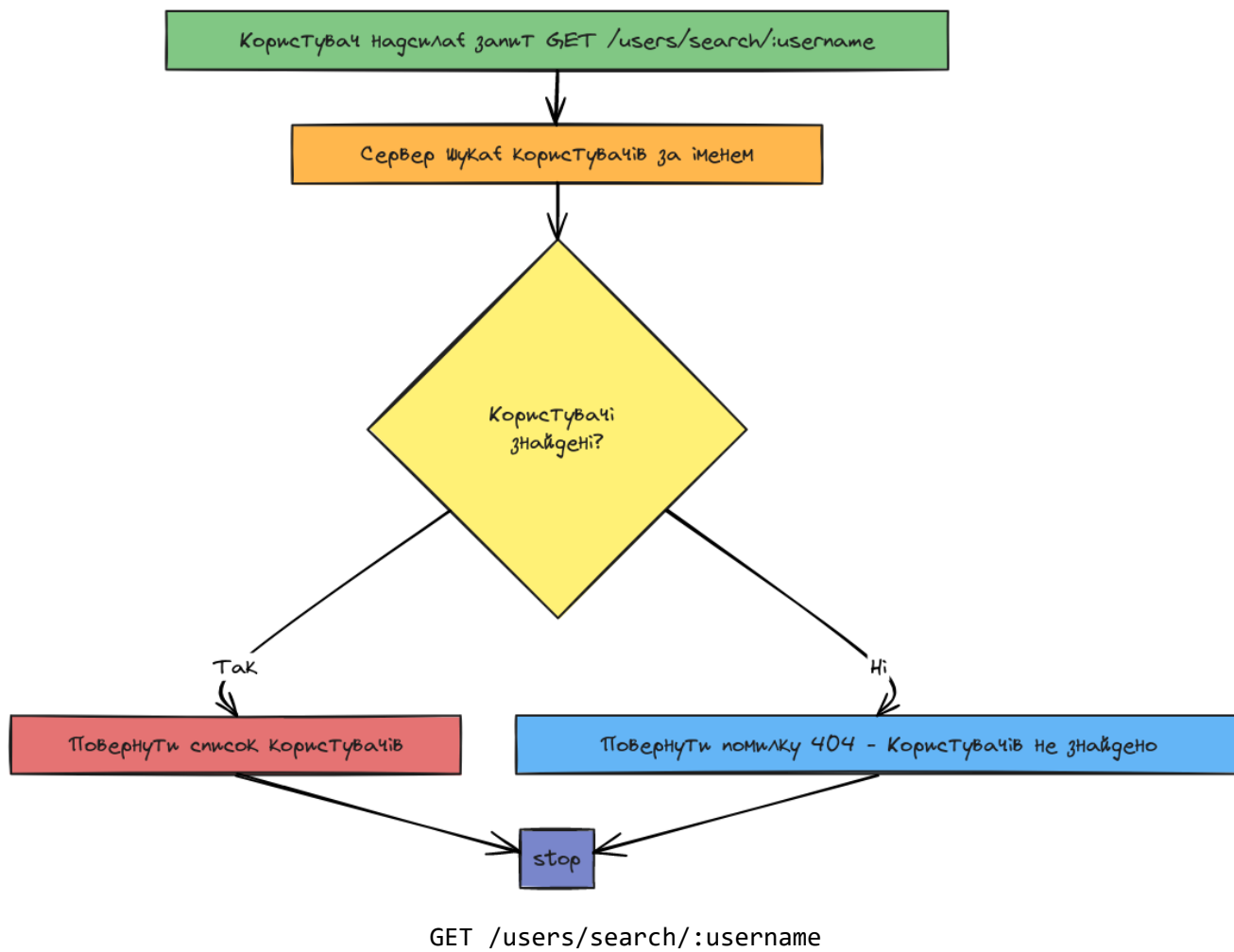
# 1. UML діаграма діяльності для серверної частини.



Шаблон UML-діаграми для запиту







## ДОДАТОК А

Фрагменти коду програми

Файл routes\reports.js

```
const express = require('express');
const router = express.Router();
const { Task } = require('../models');
const { Op } = require('sequelize');

router.get('/user/:userId', async (req, res) => {
  try {
    const userId = req.params.userId;

    const totalTasks = await Task.count({ where: { userId } });

    const completedTasks = await Task.count({
      where: { userId, isCompleted: true },
    });

    const pendingTasks = totalTasks - completedTasks;

    const overdueTasks = await Task.findAll({
      where: {
        userId,
        dueDate: { [Op.lt]: new Date() },
        isCompleted: false,
      },
    });

    const completionRate = totalTasks > 0 ? (completedTasks / totalTasks) * 100 : 0;

    res.json({
      totalTasks,
      completedTasks,
      pendingTasks,
      overdueTasks: overdueTasks.map(task => ({
        id: task.id,
        title: task.title,
        dueDate: task.dueDate,
      })),
      completionRate: `${completionRate.toFixed(2)}%`,
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.get('/user/:userId/overdue', async (req, res) => {
  try {
    const userId = req.params.userId;
```

```

const overdueTasks = await Task.findAll({
  where: {
    userId,
    dueDate: { [Op.lt]: new Date() },
    isCompleted: false,
  },
});

res.json(
  overdueTasks.map(task => ({
    id: task.id,
    title: task.title,
    dueDate: task.dueDate,
  })))
);
} catch (error) {
  res.status(500).json({ error: error.message });
}
});

router.get('/user/:userId/completion-rate', async (req, res) => {
  try {
    const userId = req.params.userId;

    const totalTasks = await Task.count({ where: { userId } });
    const completedTasks = await Task.count({
      where: { userId, isCompleted: true },
    });

    const completionRate = totalTasks > 0 ? (completedTasks / totalTasks) * 100 : 0;

    res.json({ completionRate: `${completionRate.toFixed(2)}%` });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;

```

### Файл controllers\User.js

```

const express = require('express');
const bcrypt = require('bcrypt'); // Імпорт bcrypt
const router = express.Router();
const { User } = require('../models');

// Отримати всіх користувачів
router.get('/', async (req, res) => {
  try {
    const users = await User.findAll();
    res.json(users);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

```

```

    }
  });

  // Отримати користувача за ID
  router.get('/:id', async (req, res) => {
    try {
      const user = await User.findByPk(req.params.id);
      if (!user) {
        return res.status(404).json({ message: 'User not found' });
      }
      res.json(user);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  // Створити нового користувача
  router.post('/', async (req, res) => {
    try {
      const { username, email, password } = req.body;

      // Хешування пароля
      const hashedPassword = await bcrypt.hash(password, 10); // 10 - кількість раундів
      const salt

      const newUser = await User.create({
        username,
        email,
        passwordHash: hashedPassword, // Зберігаємо хеш замість пароля
      });
      res.status(201).json(newUser);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  // Оновити інформацію користувача
  router.put('/:id', async (req, res) => {
    try {
      const user = await User.findByPk(req.params.id);
      if (!user) {
        return res.status(404).json({ message: 'User not found' });
      }
      const { username, email, password } = req.body;

      // Якщо переданий новий пароль, хешуємо його
      if (password) {
        user.passwordHash = await bcrypt.hash(password, 10);
      }

      user.username = username || user.username;
      user.email = email || user.email;
      await user.save();
      res.json(user);
    } catch (error) {

```

```

    res.status(500).json({ error: error.message });
  }
});

// Видалити користувача
router.delete('/:id', async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    await user.destroy();
    res.json({ message: 'User deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Пошук користувачів за іменем
router.get('/search/:username', async (req, res) => {
  try {
    const users = await User.findAll({
      where: {
        username: req.params.username,
      },
    });
    if (users.length === 0) {
      return res.status(404).json({ message: 'No users found' });
    }
    res.json(users);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;

```